

5 : The Processor (Pipelined)

Rajesh Panicker,
NUS

CG3207

Acknowledgement :

- Text by Harris and Harris and companion materials
- Text by Patterson & Hennessy and companion materials by Mary Jane Irwin, PSU

V2:

Additional info / clarifications / enhancements in

purple colour

Corrections in red (in slides 2, 24, 25, 34)

Instruction Critical Paths

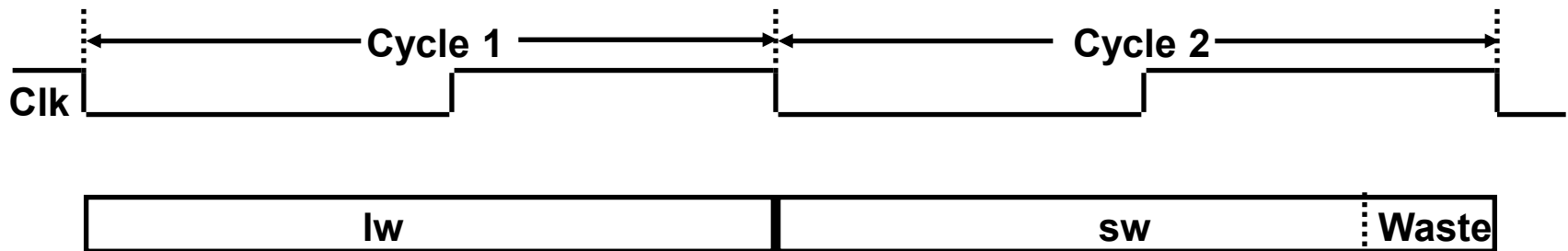
- What is the clock cycle time assuming negligible delays for muxes, control unit, extend, PC read/write, wires, setup and hold times except:
 - Instruction and Data Memory (200 ps)
 - ALU (120 ps)
 - Adders (75 ps)
 - Register File access (reads – 100 ps, writes – 60 ps)

Instr.	I Mem	Reg Rd	ALU Op	D Mem	Reg Wr	PC Incr	Total
DP	200	100	120		60		480
lw	200	100	120	200	60		680
sw	200	100	120	200			620
beq	200	100	120			75	495
jatl	200					75	275

Note : Addition for PC_IN is done in parallel with IMem access and takes even less time for all except beq

Single Cycle Disadvantages

- Uses the clock cycle inefficiently – the clock cycle must be timed to accommodate the **slowest** instruction
 - especially problematic for more complex instructions like floating point multiply



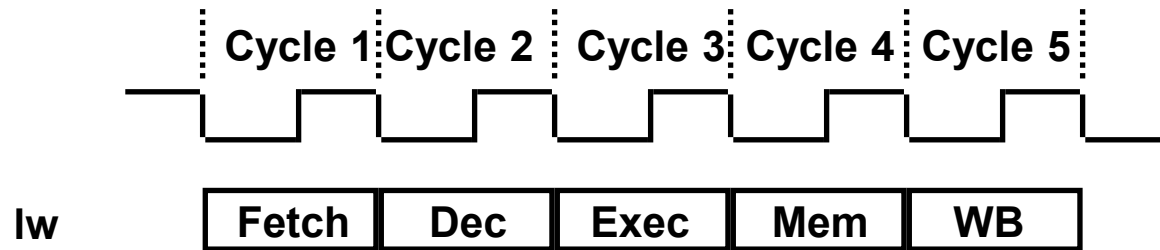
- May be wasteful of area since some functional units (e.g., adders) must be duplicated since they can not be shared during a clock cycle



How Can We Make It Faster?

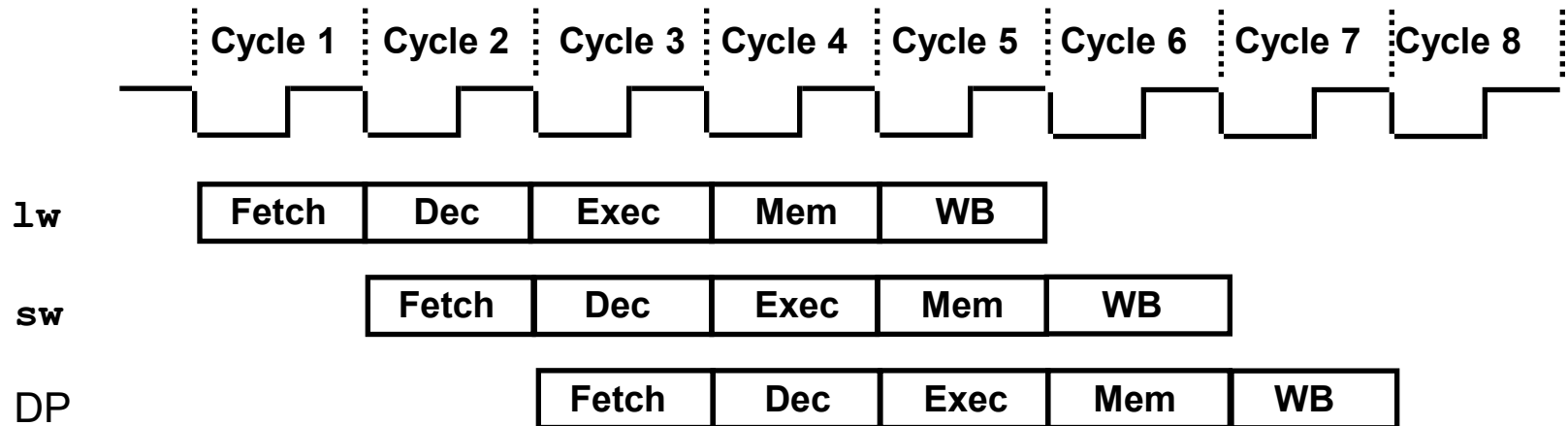
- Start fetching and executing the next instruction before the current one has completed
 - **Pipelining** – (all?) modern processors are pipelined for performance
- Remember *the* performance equation
$$\text{CPU time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$$
- Under *ideal* conditions and with a large number of instructions, the speedup from pipelining is approximately equal to the number of pipe stages
 - A five-stage pipeline is nearly five times faster because the CCT is nearly one fifth
 - Improves *throughput* - total amount of work done in a given time
 - Does NOT reduce instruction *latency* - time from the start of an instruction to its completion

The Five Stages of Load Instruction



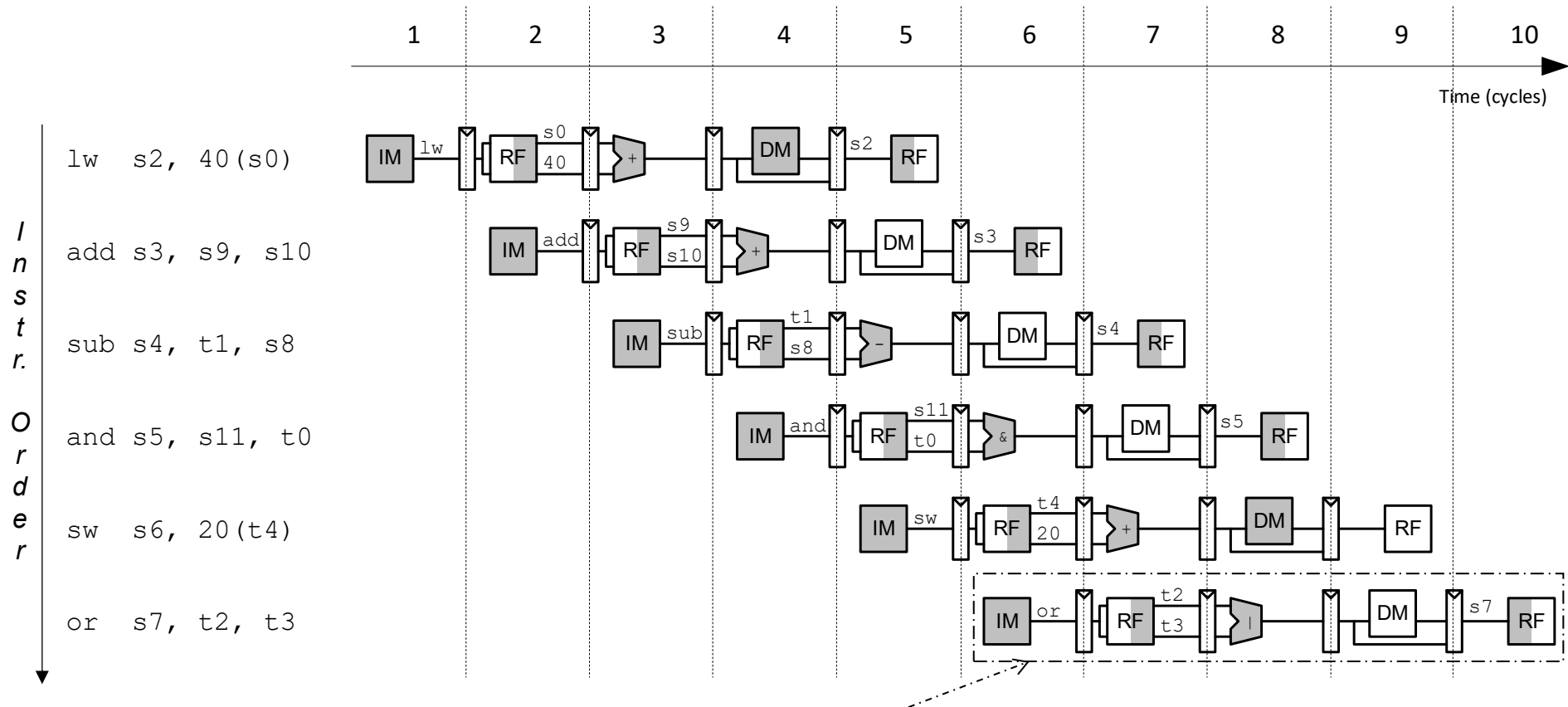
- Fetch: Instruction Fetch and Update PC
- Dec: Registers Fetch and Instruction Decode
- Exec: Execute DP-type; calculate memory address
- Mem: Read/write the data from/to the Data Memory
- WB: Write the result data into the register file

A Pipelined Processor



- Pipeline rate limited by the slowest pipeline stage
 - Some stages don't need the whole clock cycle (e.g., WB) - unbalanced pipe stages makes for inefficiencies
- The time to *fill* pipeline and time to *drain** can impact speedup for deep pipelines and short code runs
 - *Branch mispredictions, exceptions, ..
- For some instructions, some stages are wasted cycles (i.e., nothing is done during that cycle for that instruction)
 - e.g., sw in WB, beq and DP in Mem

Abstract Pipeline Diagram



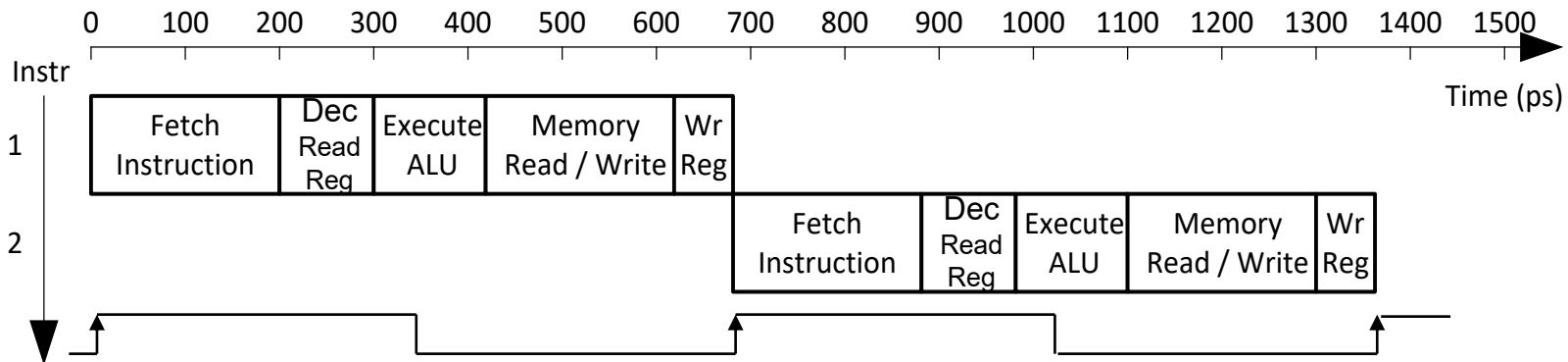
Abstract pipeline diagram

- Once the pipeline is full, (ideally) one instruction is completed every cycle, so CPI = 1

Single Cycle versus Pipeline

Single-Cycle

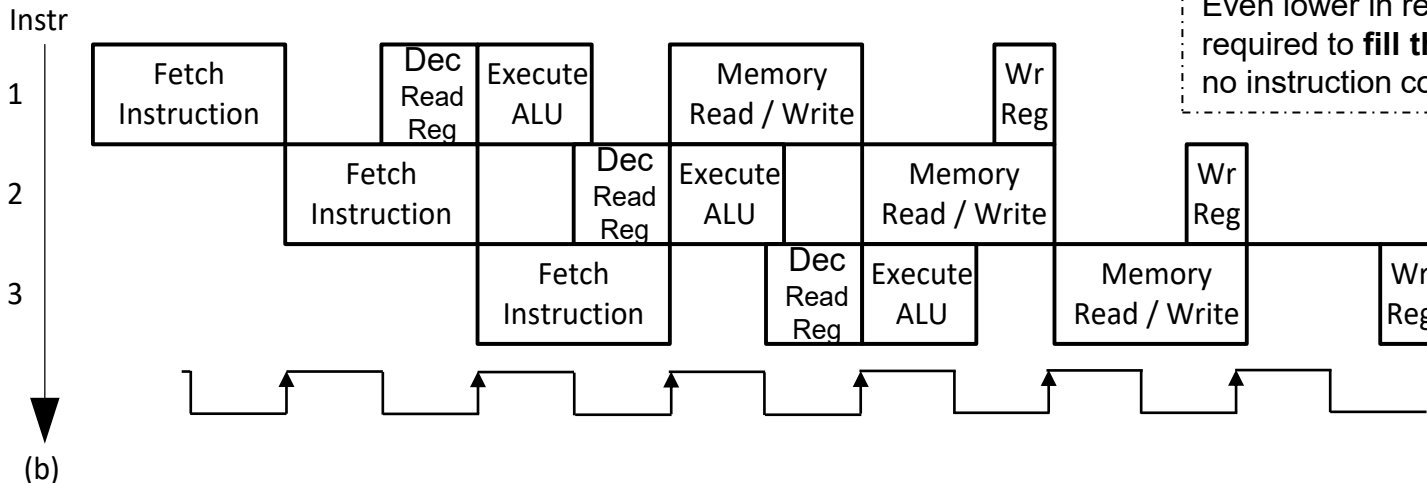
#instructions/sec = $1/680\text{ps} = 1.47$ Billion



Speedup = $5/1.47 = 3.4$
Pipeline stages not balanced =>
Speedup less than ideal value of 5.

Pipelined

#instructions/sec = $1/200\text{ps} = 5$ Billion



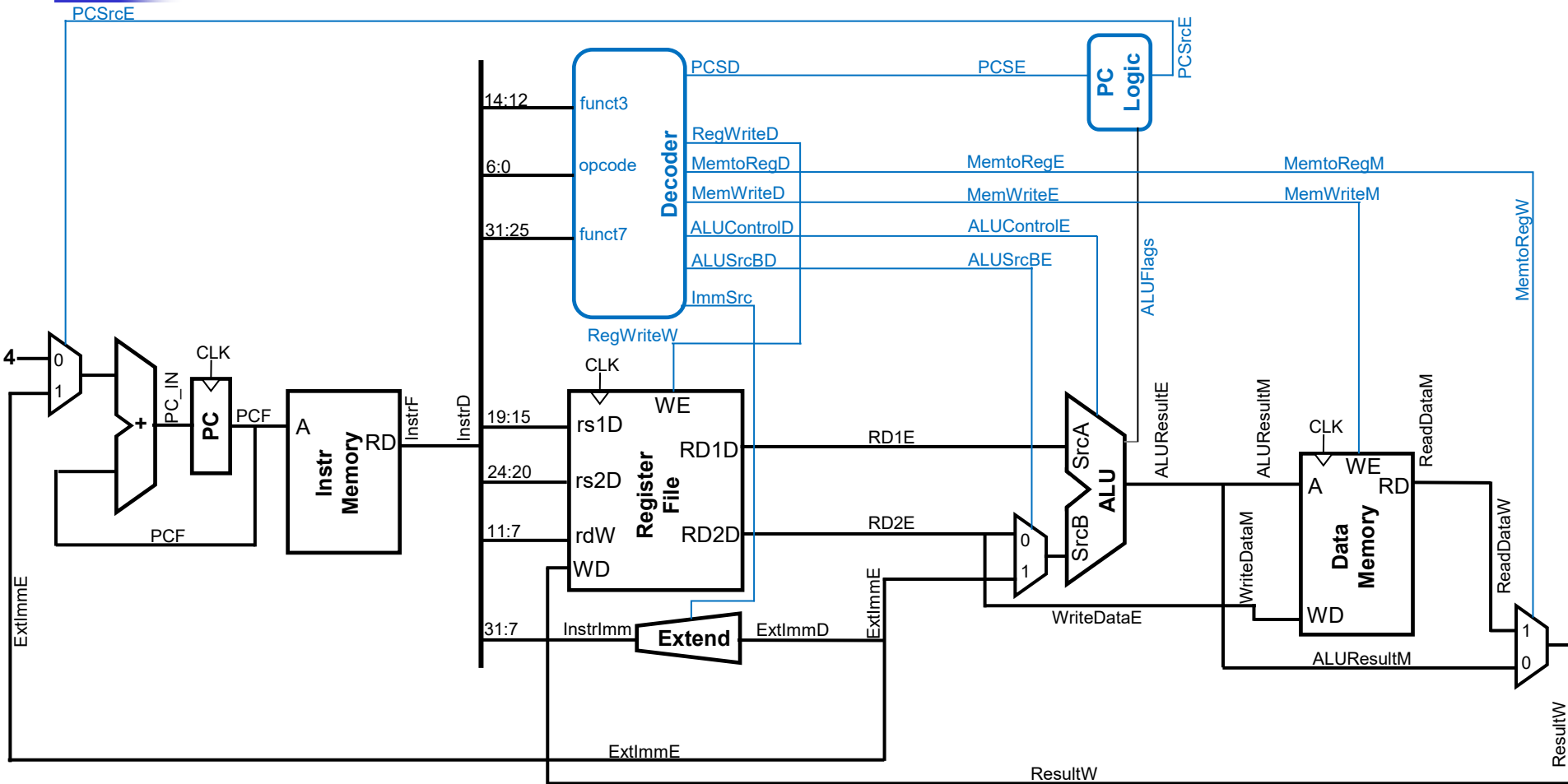
Even lower in reality - some cycles required to **fill the pipeline** during which no instruction completes execution



Pipelined Processor

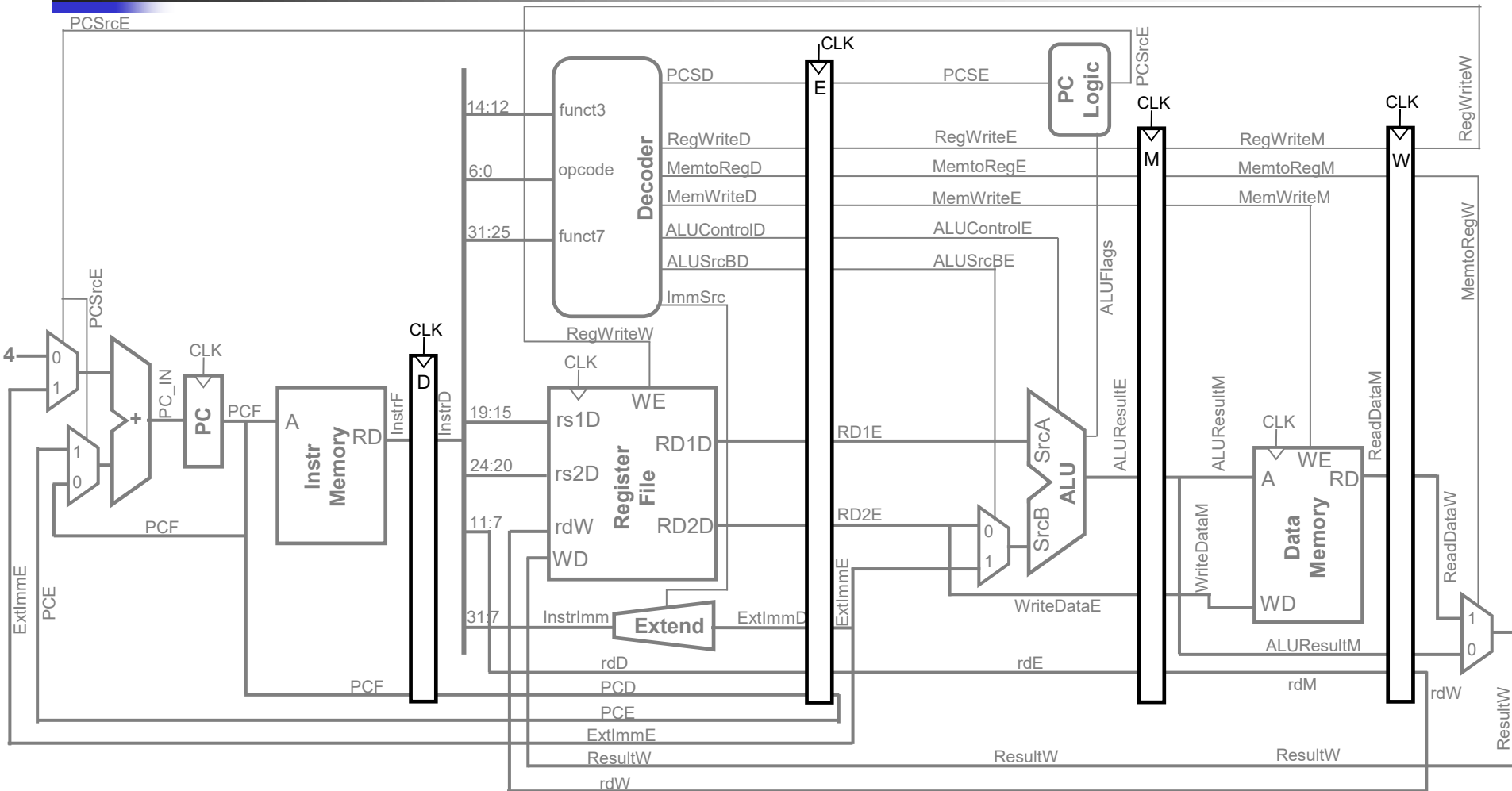
- State/pipeline registers between pipeline stages to isolate them
 - Values that go from one stage to the next passed through (delayed by) a pipeline register (and written with an appropriate suffix)
- All information related a particular instruction should be delayed equally
 - rdW, RegWriteW should be delayed to arrive at the same time as ResultW. rdD and RegWriteD should not be used directly!
 - When PCSrcE = 1, PC_IN = PCE + ExtImmE : Info is from E stage. Else, PC_IN = PCF+4 : info is from F stage
 - Choosing PCF vs PCE needs a multiplexer controlled by PCSrcE
 - The above two data movements against the 'forward' flow causes trouble. We will see soon!
- Same control unit as single-cycle processor
- Control delayed to proper pipeline stage, and dropped when it is not required in a following stage

Single-Cycle Processor



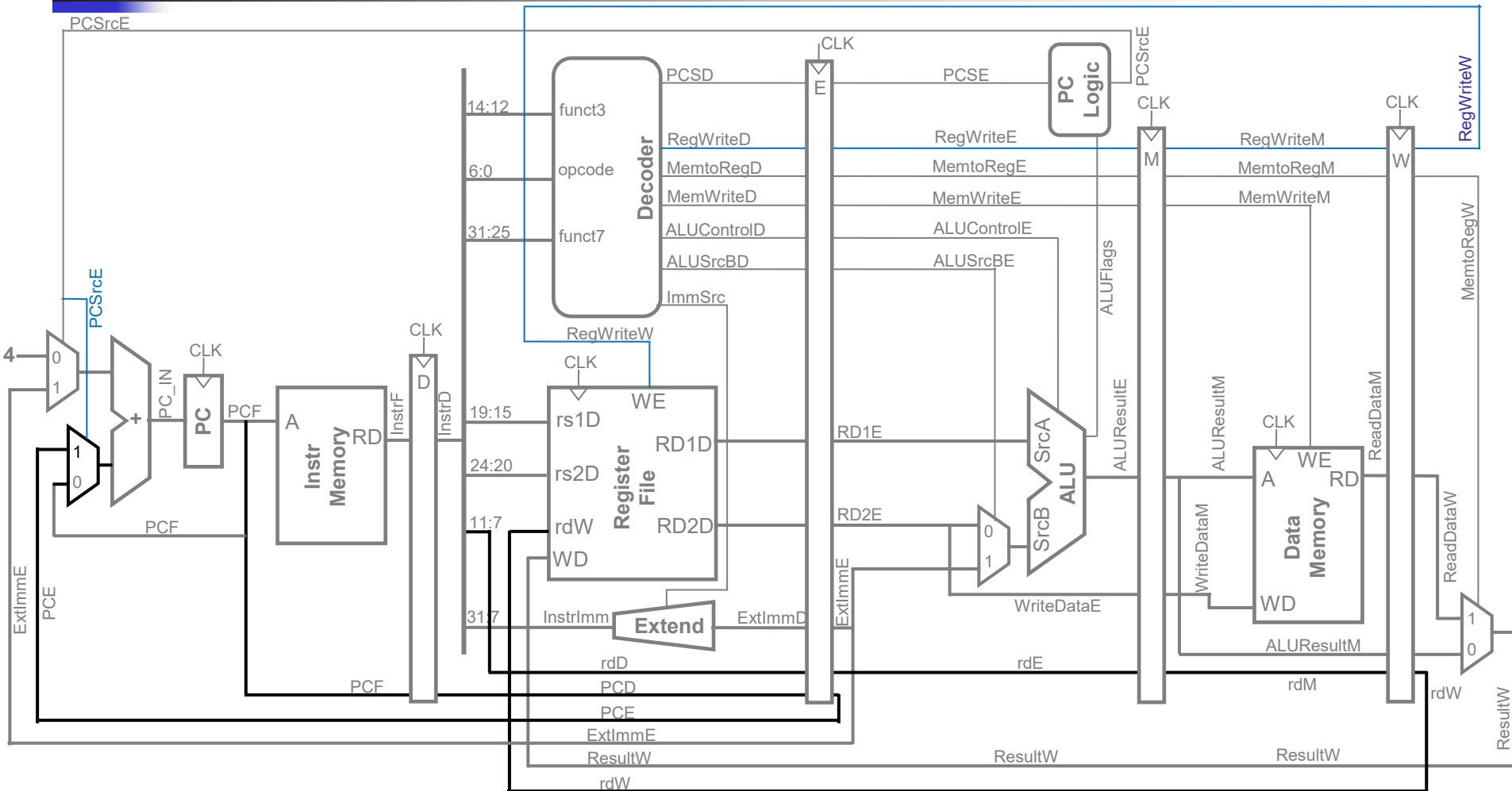
Note: Pipeline stage suffixes are not relevant here as we have not done pipelining yet

Pipelined Processor – Pipeline Registers



- Registers between each pipeline stage to isolate them and reduce the critical path

Pipelined Processor





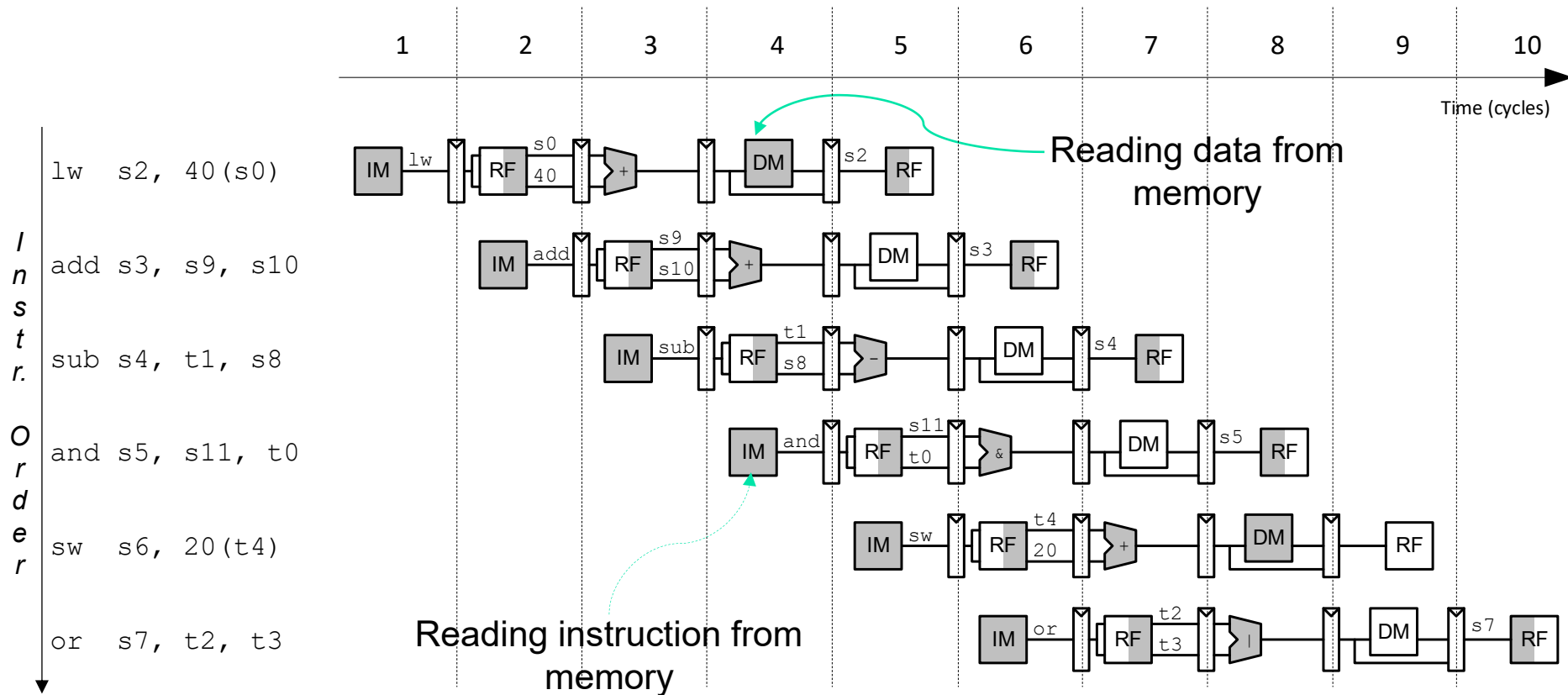
In fact, from a timing perspective, moving PC logic to M stage (which means using PCSrcM, ExtImmM, PCM used for branching) is better. It helps make E stage shorter, which could reduce the critical path delay if stage E is the longest. However, moving branch to M has some disadvantages too as we will see later.



Can Pipelining Get Us Into Trouble?

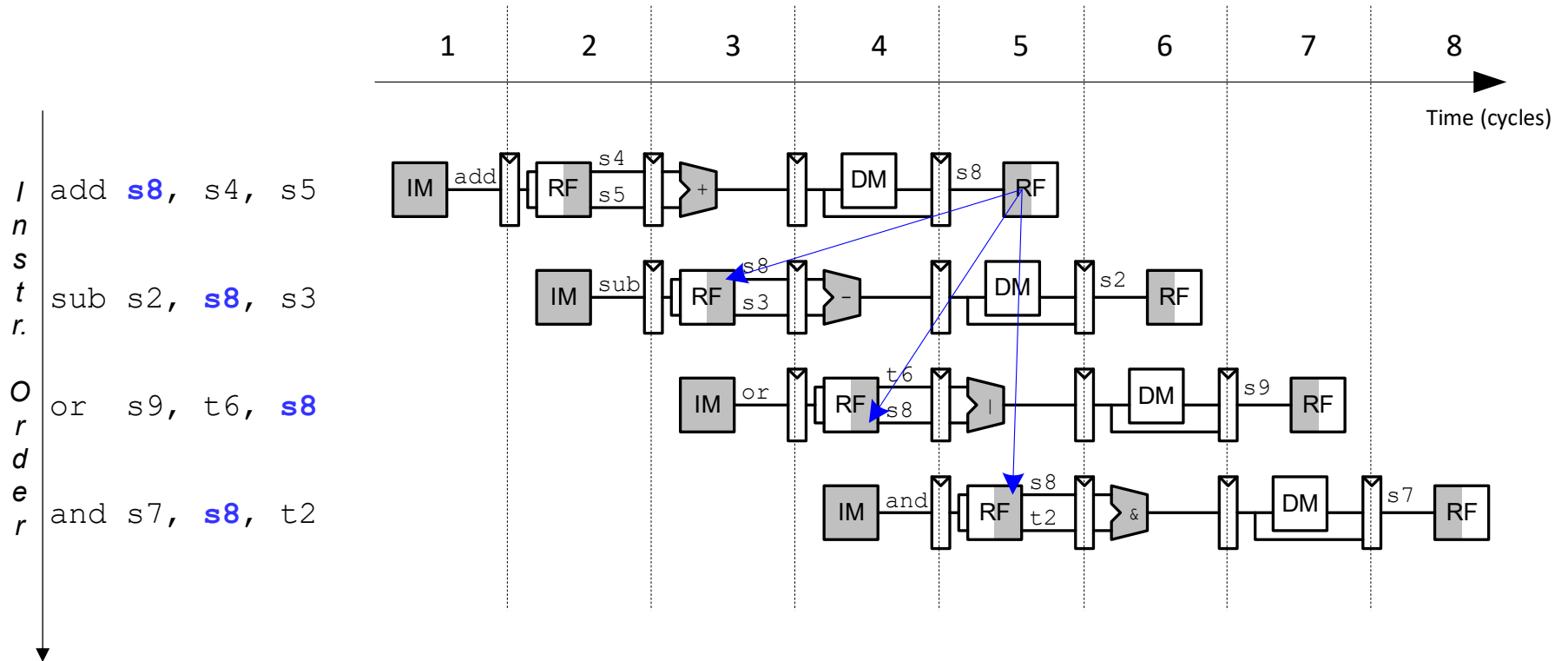
- Yes: **Pipeline Hazards**
 - **structural hazards**: attempt to use the same resource by two different instructions at the same time
 - **data hazards**: attempt to use data before it is ready
 - An instruction's source operand(s) are produced by a prior instruction still in the pipeline
 - **control hazards**: attempt to make a decision about program control flow before the condition has been evaluated and the new PC target address calculated
 - Branch/jump instructions (jumps are also henceforth referred to generically as 'branches'), exceptions
- Can usually resolve hazards by **waiting**
 - Compiler (**software**) or pipeline control (**hardware**) must detect the hazard
 - and take action to resolve hazards

A Single Memory Would Be a Structural Hazard



- Fix with separate instr and data memories (IM and DM)
 - Or at least separate caches – problem arises only when there is a simultaneous instruction and data cache miss

Data Hazard



- Also known as RAW (read after write) hazard or true data dependency
- Occurs very frequently in practice -> represents the flow of information in the program



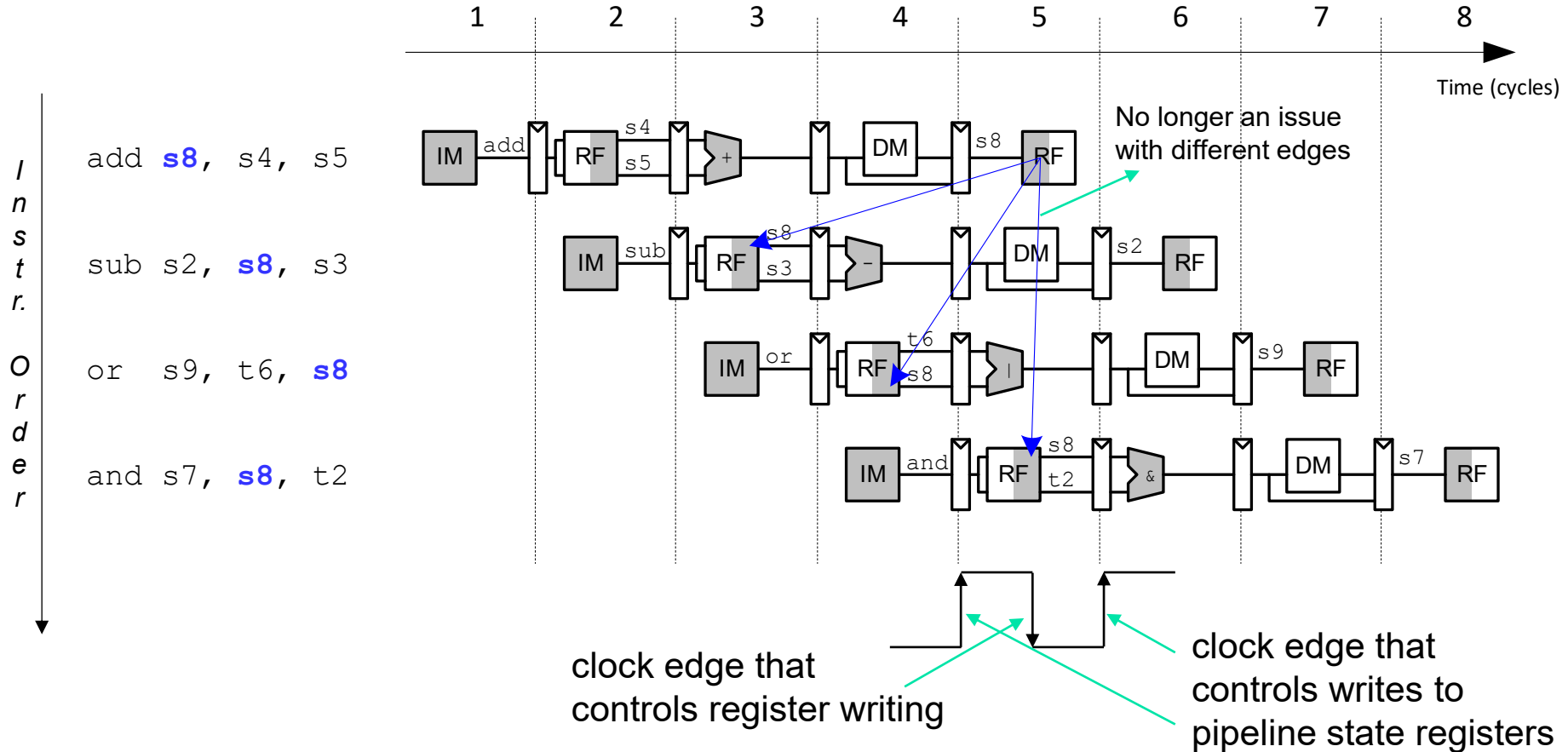
Handling Data Hazards

- Write register file and pipeline registers at different edges of the clock
- Insert `NOPS` in code at compile time
 - The assembly instruction `NOP` expands to `addi x0, x0, 0`
- Rearrange code at compile time
- Forward data at run time
- Stall the processor at run time

Handling Data Hazards

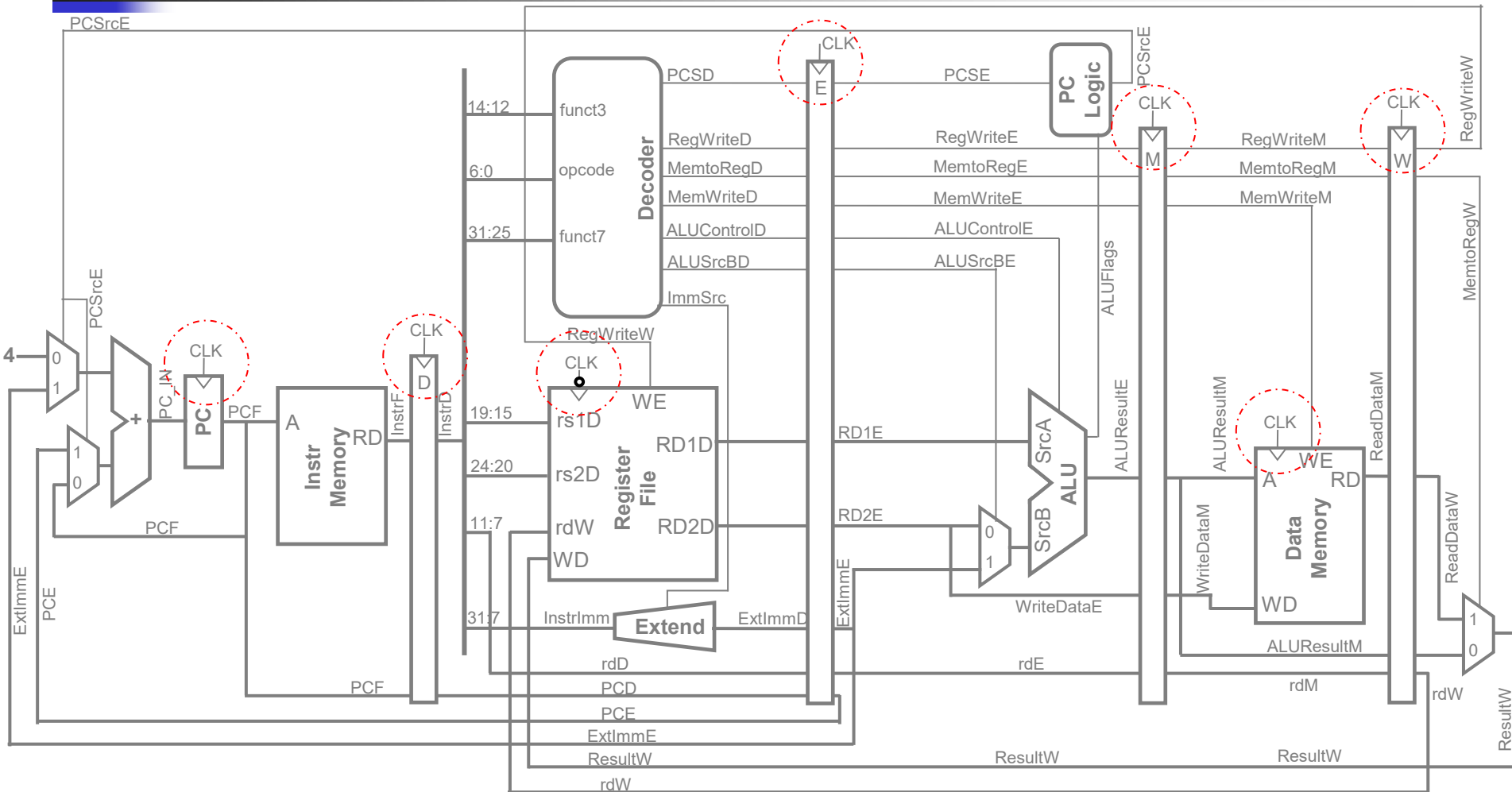
- Write register file and pipeline registers at different edges of the clock
- Insert NOPS (`addi x0, x0, 0`) in code at **compile time**
 - Insert enough NOPs for result to be ready
 - NOPs waste time => No useful work is done
 - NOPs waste code memory => makes the code bulkier / bloated
 - Compiler needs to know the microarchitecture to know the number of NOPs => code is not very portable
- Rearrange code at **compile time**
 - Move independent useful instructions forward
 - Might not be possible all the time
- Stall the processor at **run time**
 - Impacts performance, needs additional hardware (**not covered**)
- Forward data at **run time**
 - Good performance, needs even more hardware (**not covered**)

Data Hazard : Using Different Clock Edges



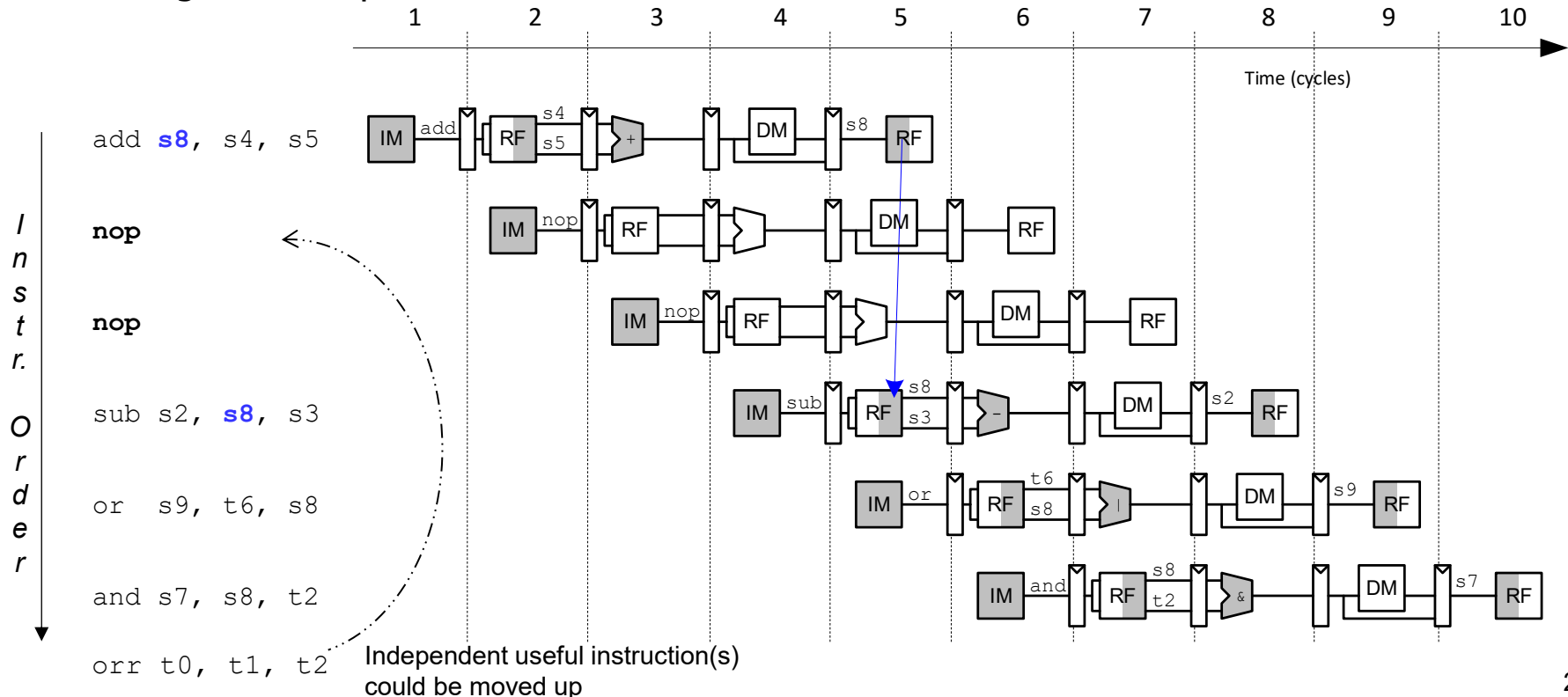
- Reading in the second half of the cycle and writing in the first half helps for instructions which are spaced by at least 2 instructions

Data Hazard : Using Different Clock Edges

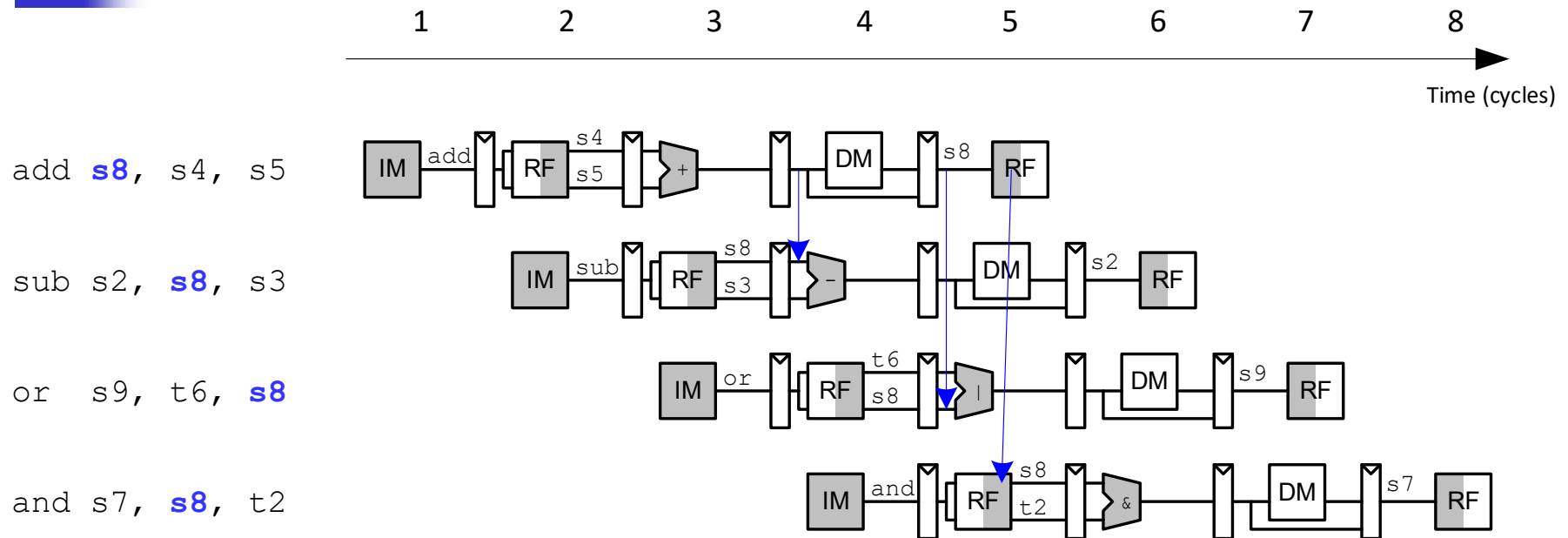


Data Hazard : Inserting NOPs

- Insert enough NOPs for result to be ready
 - NOPs waste time => CPI is **greater** than the ideal value of 1
 - NOPs waste code memory => makes the code bulkier / bloated
 - Compiler needs to know the microarchitecture => code is not very portable
- Or move independent useful instructions forward
 - Might not be possible all the time



Data Hazard : Data Forwarding

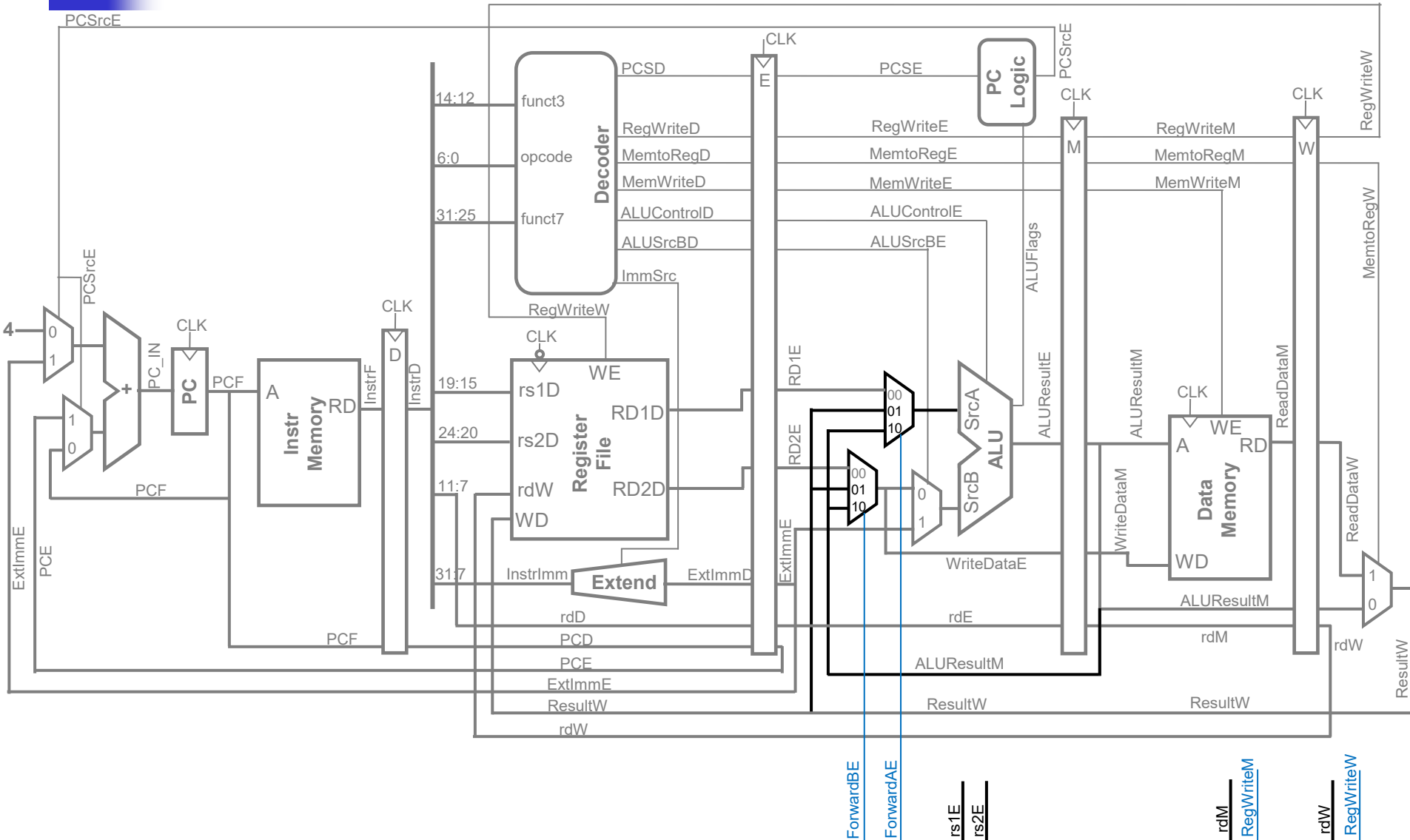


- Data is available on some pipeline register before it is written back to the register file (RF)
 - Take it from where it is present, pass it to where it is needed
- Check if register read by the instruction which is currently in Execute stage matches register written by the instruction which is currently in Memory or Writeback stage
 - If so, forward result – the input to ALU now comes from M or W stage rather than E register

Data Forwarding ...

- Take the result from the earliest stage / point (latest in the code) that it exists in **any** of the **later** (earlier in the code) pipeline state registers and forward it to the functional units (e.g., ALU) that need it that cycle
- For ALU functional unit: the inputs can come from any pipeline register rather than just from E by
 - adding multiplexers to the inputs of the ALU
 - connecting ALUResultM and ResultW to SrcAE and SrcBE ALU mux inputs
 - adding the proper control hardware to control the new muxes
- Other functional units may need similar forwarding logic (e.g., the Data Memory)
- Can achieve a CPI of 1 even in the presence of data dependencies
- Data forwarding is also called **Bypassing**
 - The usual register file write and then read process for data access is bypassed
 - Register is still written, for future reads

Data Forwarding : Circuitry



Data Forwarding : Conditions

- **Case 1:** Execute stage rs1 or rs2 matches Memory stage Rd?
Forward from Memory stage
- **Case 2:** Execute stage rs1 or rs2 matches Writeback stage Rd?
Forward from Writeback stage
- **Case 3:** Otherwise use value read from register file (as usual)

Equations for Rs1:

```
if      (rs1E == rdM) & RegWriteM & (rdM* != 0)    // Case 1
        ForwardAE = 10
else if (rs1E == rdW) & RegWriteW & (rdW* != 0)    // Case 2
        ForwardAE = 01
else    ForwardAE = 00                             // Case 3
```

ForwardBE equations are similar (replace **rs1E** with **rs2E**)

*Checking rs1E != 0 is also fine

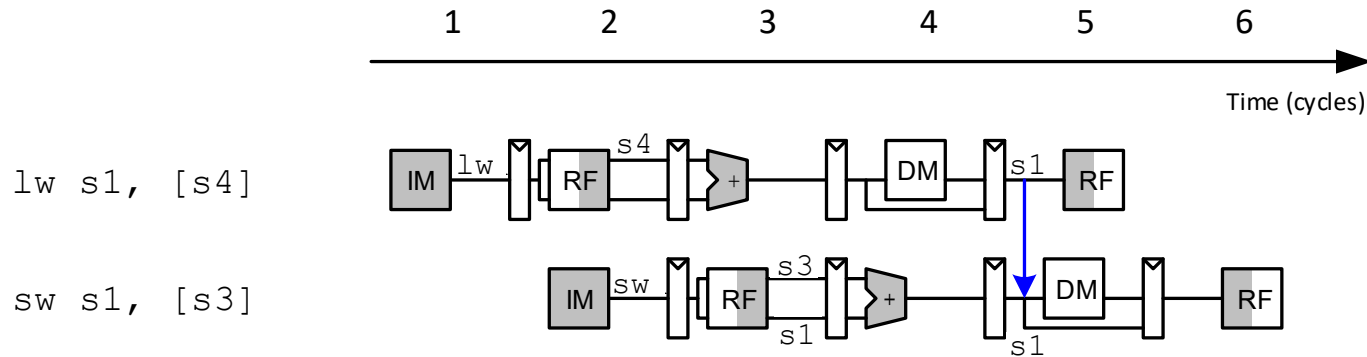
Forward only if the instruction in M / W really writes to a register that is not x0.

If rdM / rdW == 0, x0 does not get updated by the instruction in M / W stage though RegWrite is 1.

Branch, sw do not write to any register - rd field is used for immediate. RegWriteM/W guards against spurious matches.

Each comparison requires a 5-bit comparator. Total 6 x 5-bit comparators.

Data Forwarding (Memory-Memory copy)



- Check if the register used in Memory stage by the `sw` instruction (i.e., `rs2` of `sw`) matches register written by `lw` in Writeback stage (i.e., `rd` of `lw`)
- If so, forward result

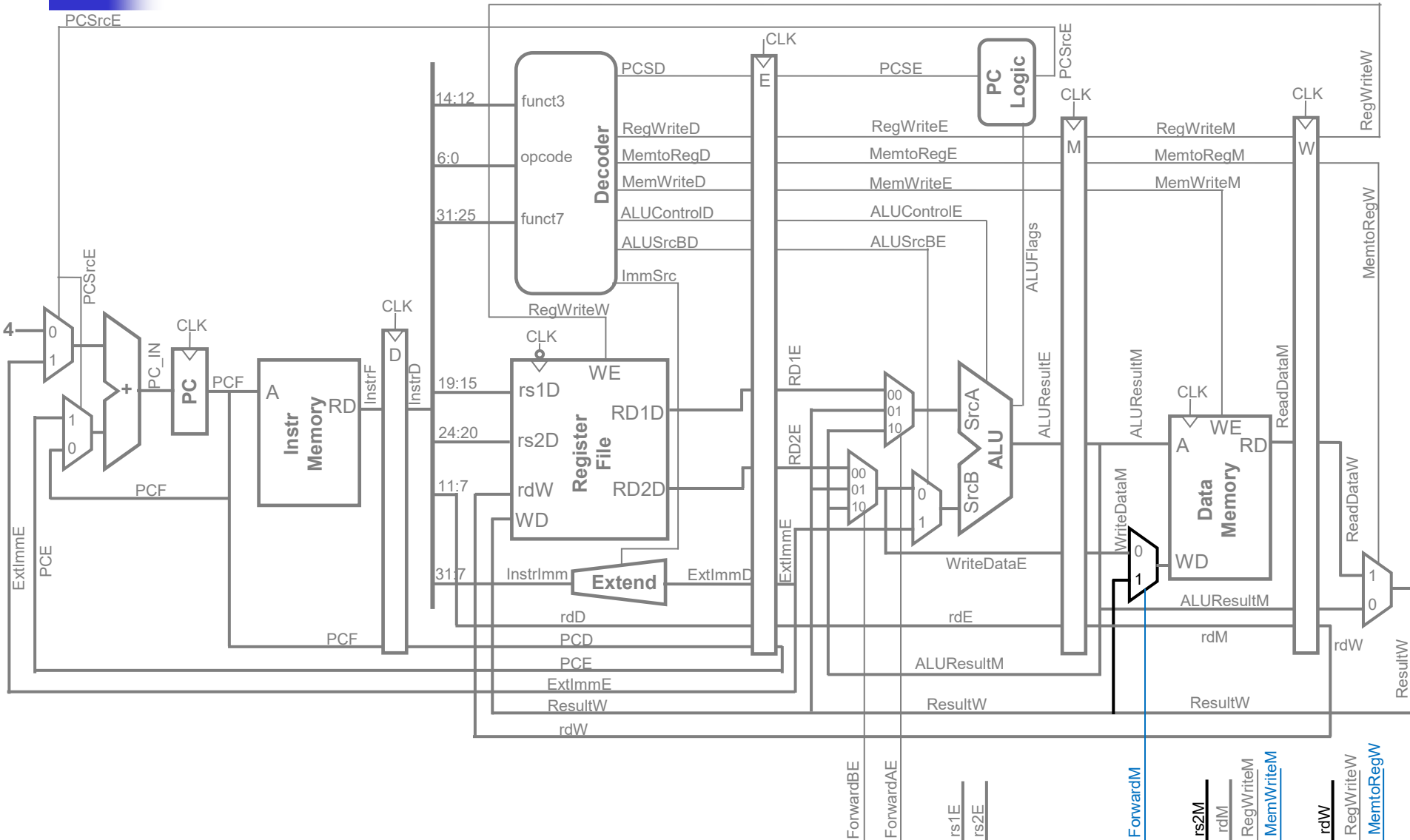
`sw` writes `[rs2]` into the memory

Check if M has `sw` and W has `lw` with a non-`x0` destination

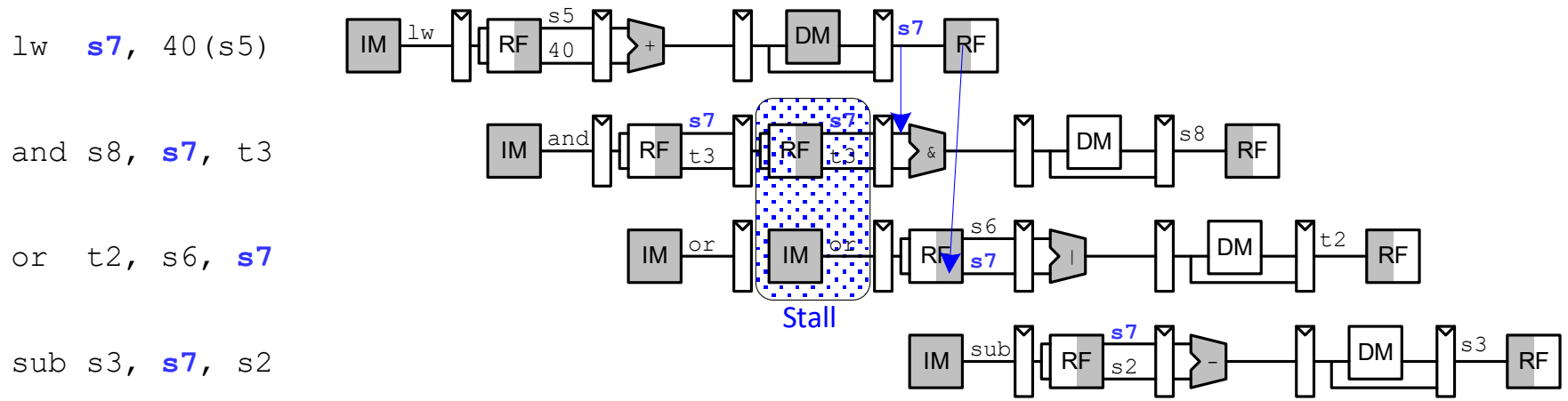
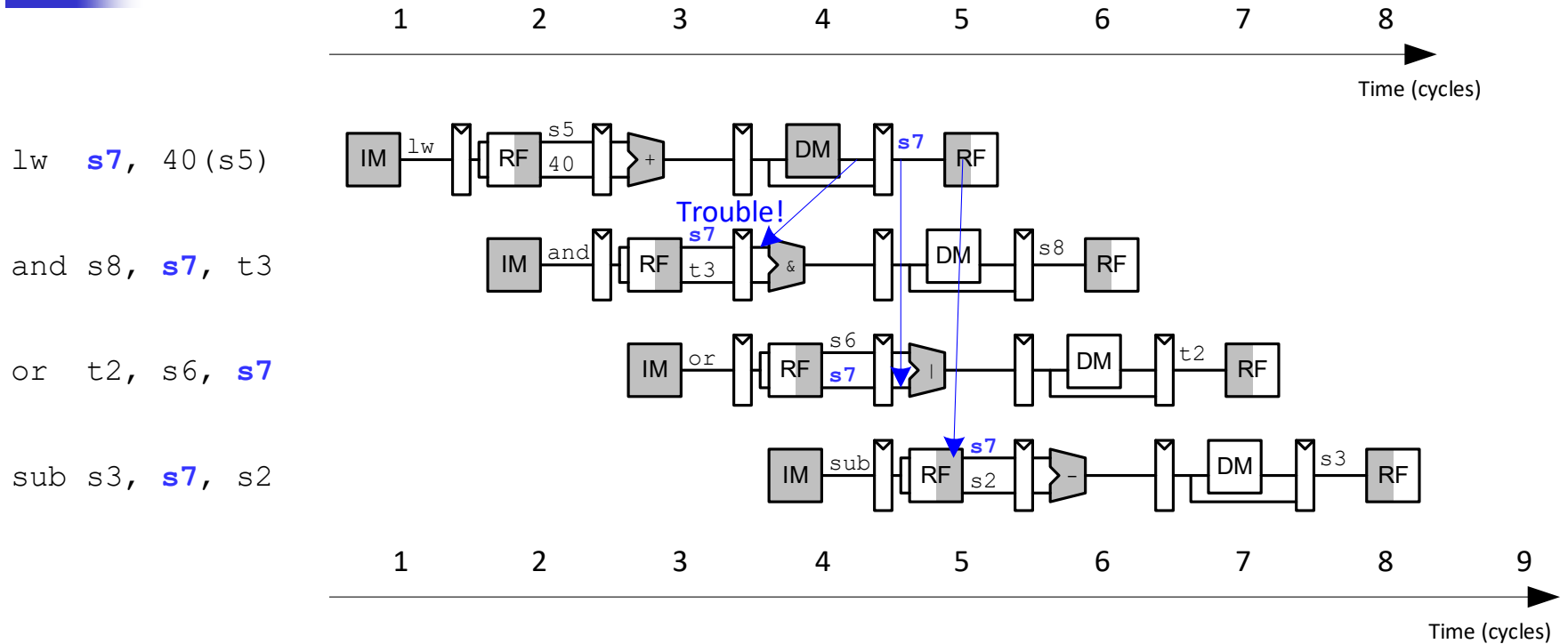
$$\text{ForwardM} = (\text{rs2M} == \text{rdW}) \& \text{MemWriteM} \& \text{MemtoRegW} \& (\text{rdW} \neq 0)$$

This is assuming that `MemtoReg` is asserted only for `lw`. If that is not the case (e.g., `MemtoReg` is 'don't care' or asserted for `sw`), need `& RegWriteW` in the above expression

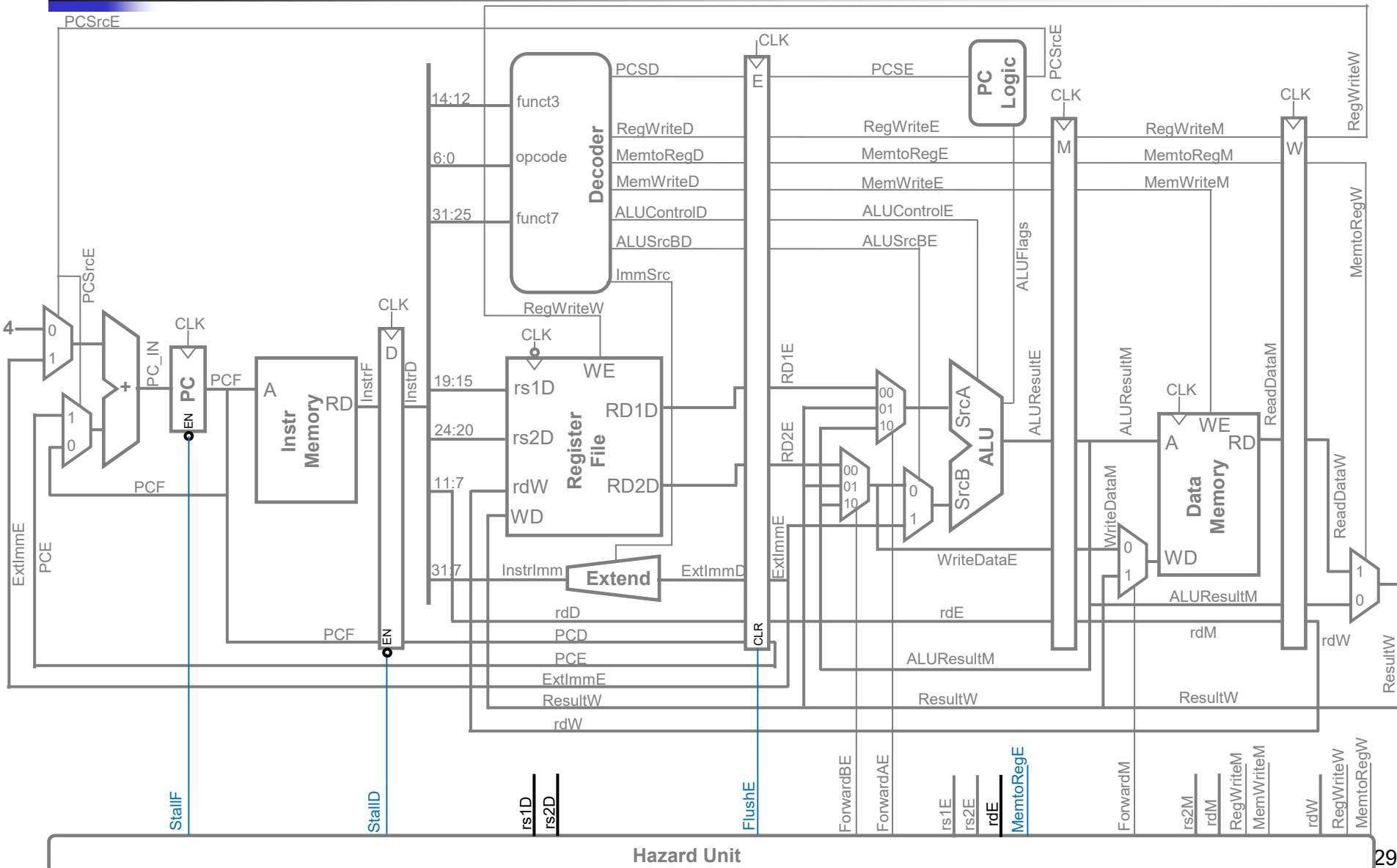
Data Forwarding : Circuitry (Mem-Mem copy)



Load and Use Hazard : Stalling



Load and Use Hazard : Stalling Circuitry



Load and Use Hazard : Condition

- Is either source register of instruction in the Decode stage the same as the destination of the instruction which is currently in the Execute stage? *AND*
- Is the instruction in the Execute stage *lw* ? If yes, stall

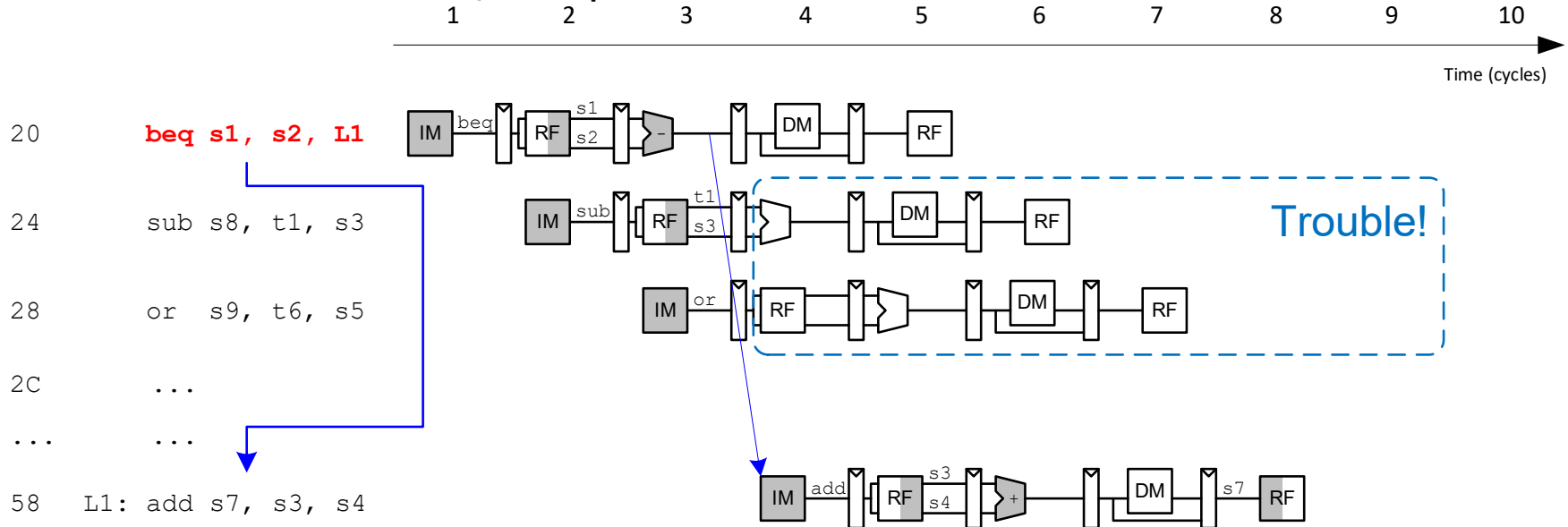
$$lwStall = ((rs1D == rdE) \mid (rs2D == rdE)) \& MemtoRegE$$

$$StallF = StallD = FlushE = lwStall$$

- After the 1 cycle stall, the forwarding circuitry (W to E) will take care of delivering the correct data
- Unnecessary stalls can be avoided in the following cases by adding appropriate exception conditions (Omitted here for simplicity)
 - J (*jal*) and U (*lui, auipc*) instructions do not use *rs1*.
 - J, U, I (*lw, DP Imm*) instructions do not use *rs2*.
 - Mem-mem copy forwarding takes care of *sw* following *lw*.
 - *rdE == 0* doesn't warrant stalling

Control Hazards

- Branch instructions, exceptions can cause control hazards



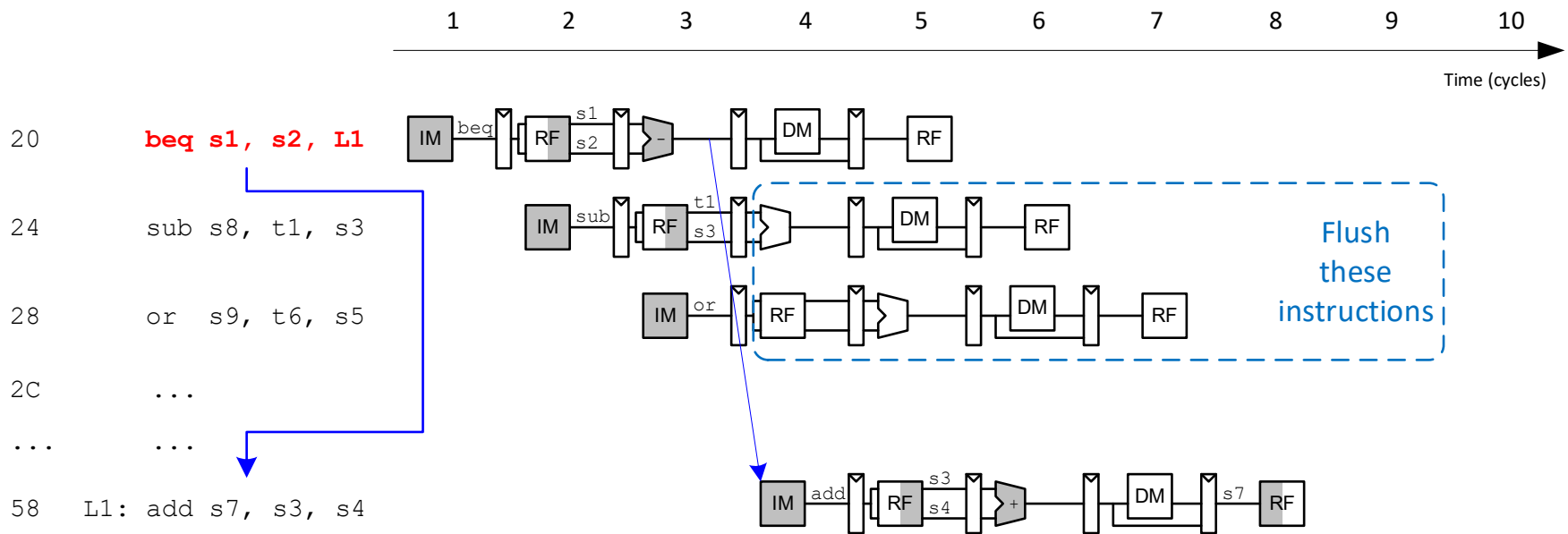
- Compile time NOPs or rearranging the code can help, but
 - NOPs reduce performance, even when the branch is not taken
 - Increases the code size and makes the code microarchitecture dependent
- Delay decision a.k.a delay slot – compiler inserts a certain number (e.g., 2) of independent (and hopefully useful, if not NOPs) instructions which are safe to execute irrespective of branch outcome
 - Specified in the ISA, so the microarchitecture designer will assume that the 2 instructions following the branch can be (and should be) executed

Control Hazards

- Compiler can also reduce control hazards through loop unrolling, function inlining, etc. which reduces branches
- Hardware-based approaches
 - Stall until branch decision and BTA are available (impacts CPI)
 - Early branching - move decision point as early in the pipeline as possible, thereby reducing the number of stall cycles
 - e.g., to decode stage, but will need additional comparison circuitry in D stage and could increase the critical path by making D stage take longer.
 - Flush instructions that entered the pipeline if the branch is taken
 - Branch prediction - predict and hope for the best !
 - Fetch and execute from both branches (if BTA is known)
 - Predicated/conditional execution, fine grained multithreading,...
- Control hazards occur less frequently than data hazards, but there is *nothing* as effective against control hazards as forwarding is for data hazards

Control Hazards - Flushing

- Branch not determined until the Execute stage of pipeline
- Two instructions after branch fetched before branch occurs

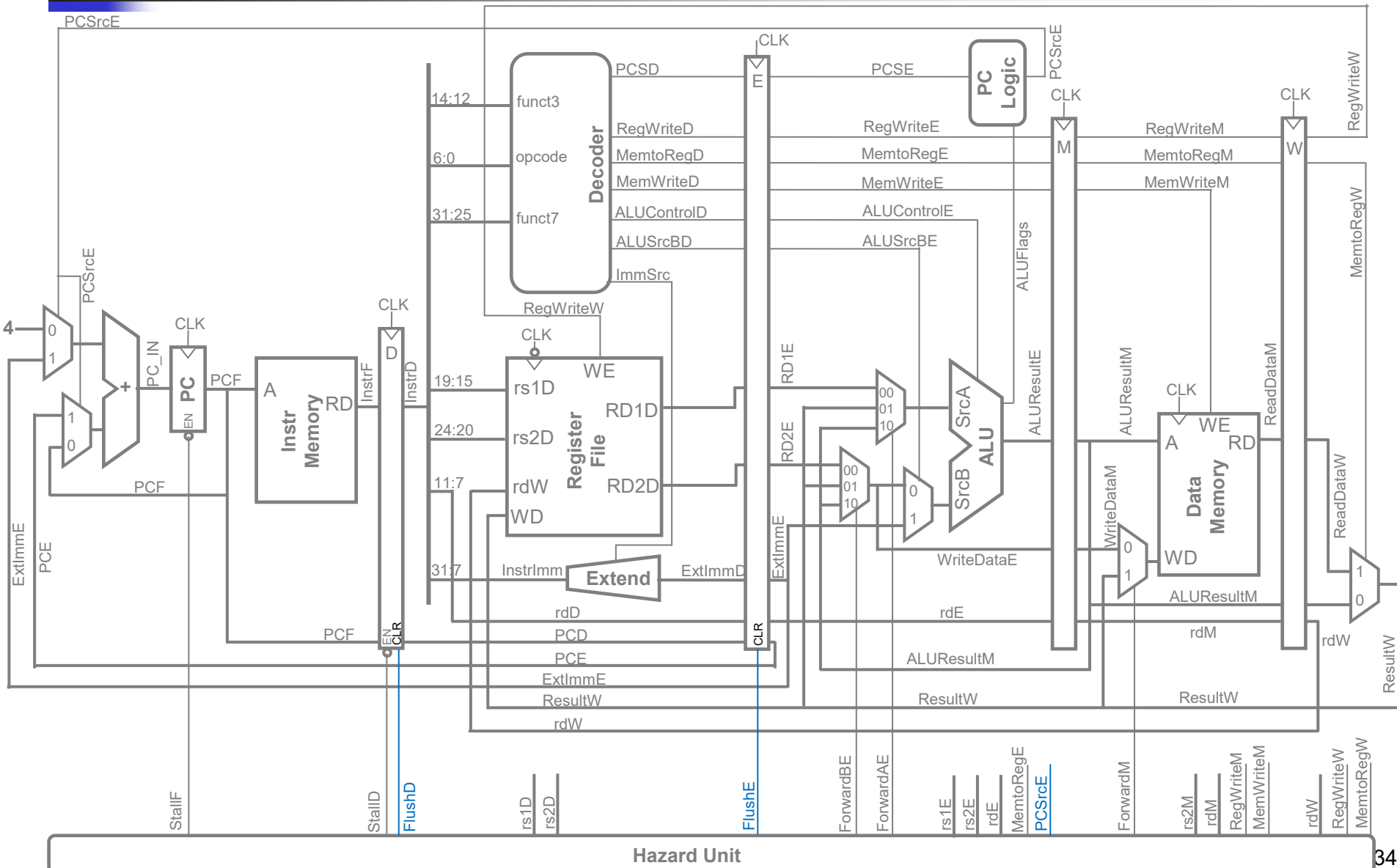


- If PCSrcE is true, flush the two earlier stages to quash the two following instructions

$$\text{FlushD} = \text{FlushE} = \text{PCSrcE}$$

$$\text{FlushE (effective)} = \text{lwstall (load and use)} \parallel \text{PCSrcE (control)}$$

Control Hazards : Flushing Circuitry



*clearing all bits work if an instruction with all 0s is decoded as NOP. Else, use clear to control a multiplexer that selects between the instruction and a hardcoded NOP intrn. In RV ISA, an instrn with all 0s is invalid.

Stall vs Flush

- Stall involves deasserting the write control of the pipeline register
- Flush involves asserting the 'clear' of the pipeline register
 - In practice, clear only those bits which can cause a change in state (PCSt^{re}E, MemWriteE, RegWriteE is sufficient for E. InstrD* for D – instrn isn't decoded yet)
 - MemtoRegE should also be cleared, so that a load and use stall is only for one cycle (if the expression for lw stall does not include RegWriteE), in one particular scenario
- If any stage is stalled, (i) the previous stages should also be stalled, and (ii) the following stage should be flushed (think load and use)
 - If (ii) is not done, the instruction in the latest stage being stalled will be executed 2 times – first using incorrect data and later using correct data (this is not ok if the instruction cause exceptions, writes to wrong locations [if sw], etc.)
 - Effectively, an NOP is inserted between the instruction in the latest stage being stalled and the instrn in the *following* pipeline stage (*earlier* in code)
 - Instructions in the stages being stalled are bounced in place, while the instructions in the later stages of the pipeline (earlier in the code) are allowed to proceed to completion normally
- If any stage is flushed without stalling the previous stage, the instruction which is supposed to go into the flushed stage is **replaced** with an NOP (think branch)



W to D Forwarding

- Can we write pipeline registers and register file at the same edge, so that we can use the same clock edge for everything?
- Yes! But needs forwarding from W to D
- $\text{Forward1D} = (\text{rs1D} == \text{rdW}) \ \& \ \text{RegWriteW} \ \& \ (\text{rdW} \neq 0)$
- $\text{Forward2D} = (\text{rs2D} == \text{rdW}) \ \& \ \text{RegWriteW} \ \& \ (\text{rdW} \neq 0)$







Summary

- Almost all modern day processors use pipelining
- Pipelining doesn't help **latency** of single task, it helps **throughput** of entire workload
- Potential speedup: a CPI of 1 and a fast CCT
- Pipeline rate limited by **slowest** pipeline stage
 - Unbalanced pipe stages makes for inefficiencies
 - The time to "**fill**" pipeline and time to "**drain**" (flush) it can impact speedup for deep pipelines and short code runs
- Must detect and resolve hazards at compile time (by the compiler) or runtime (by the hardware)
 - Compile time NOPs reduce performance, increases code size and limits the portability of the code
 - Run time hazard detection requires additional hardware